

Memoria y evaluación de cuestiones teórico-prácticas. P2

Ejercicio 1). Apartado A)

Explicación de la codificación empleada: En primer lugar, hemos establecido que 1 unidad equivalga a 10 ml, de esta manera, simplificamos el espacio de búsqueda para el solver y facilita el usar solo enteros. En segundo lugar, hemos usado un enum para los ingredientes, y de esta manera poder usarlos como índices, como se hace en x, que es la variable de decisión principal, ya que en este array, cada ingrediente tendrá un valor entre 0 y el volumen total de tal manera que se cumplan las restricciones. Por otro lado, hemos usado décimas para representar los céntimos, gramos y miligramos, y de esta manera, tener siempre valores enteros. Por último, hemos establecido todas las restricciones.

Volumen total buscado (en ml)	Primera solución encontrada	Número total de soluciones	Runtime (en milisegundos) para calcular todas las posibles soluciones
200	Cantidades usadas: ----- AGUA_BASE: 0 ml JARABE_A: 0 ml JARABE_B: 120 ml CONCENTRADO_SALINO: 30 ml EXTRACTO_ENERGETICO: 40 ml AROMA_CITRICO: 10 ml Coste total = 23.0 centimos Azucar total = 16.2 g Sodio total = 82.5 mg Cafeina total = 12.0 mg	6	143
1000	Cantidades usadas: ----- AGUA_BASE: 0 ml JARABE_A: 0 ml JARABE_B: 620 ml	398	355

	CONCENTRADO_SALINO: 150 ml EXTRACTO_ENERGETICO: 200 ml AROMA_CITRICO: 30 ml Coste total = 115.8 centimos Azucar total = 83.0 g Sodio total = 415.5 mg Cafeina total = 60.0 mg		
5000	Cantidades usadas: ----- AGUA_BASE: 0 ml JARABE_A: 0 ml JARABE_B: 3140 ml CONCENTRADO_SALINO: 710 ml EXTRACTO_ENERGETICO: 1000 ml AROMA_CITRICO: 150 ml Coste total = 579.8 centimos Azucar total = 419.8 g Sodio total = 2013.5 mg Cafeina total = 300.0 mg	41031	17739
10000	Cantidades usadas: ----- AGUA_BASE: 0 ml JARABE_A: 0 ml JARABE_B: 6290 ml CONCENTRADO_SALINO: 1410 ml EXTRACTO_ENERGETICO: 2000 ml AROMA_CITRICO: 300 ml Coste total = 1159.8 centimos Azucar total = 840.8 g Sodio total = 4011.0 mg Cafeina total = 600.0 mg	320957	157000

Apartado B)

Explicación de la codificación empleada: Igual que en el apartado A, simplemente como pasamos de tener un problema CSP a uno COP, indicamos al solver que vamos a minimizar.

Volumen total buscado (en ml)	Solución óptima	Runtime (en milisegundos)
200	Cantidades usadas: ----- AGUA_BASE: 0 ml JARABE_A: 0 ml JARABE_B: 110 ml CONCENTRADO_SALINO: 30 ml EXTRACTO_ENERGETICO: 40 ml AROMA_CITRICO: 20 ml Coste total = 22.6 centimos Azucar total = 15.2 g Sodio total = 81.0 mg Cafeina total = 12.0 mg	3
1000	Cantidades usadas: ----- AGUA_BASE: 0 ml JARABE_A: 0 ml JARABE_B: 500 ml CONCENTRADO_SALINO: 160 ml EXTRACTO_ENERGETICO: 200 ml AROMA_CITRICO: 140 ml Coste total = 111.2 centimos Azucar total = 70.8 g Sodio total = 415.0 mg Cafeina total = 60.0 mg	352
5000	Cantidades usadas: ----- AGUA_BASE: 0 ml JARABE_A: 0 ml JARABE_B: 2460 ml CONCENTRADO_SALINO: 770 ml EXTRACTO_ENERGETICO: 1000 ml AROMA_CITRICO: 770 ml Coste total = 553.8 centimos Azucar total = 350.6 g Sodio total = 2016.5 mg Cafeina total = 300.0 mg	728

10000	Cantidades usadas: ----- AGUA_BASE: 0 ml JARABE_A: 0 ml JARABE_B: 4910 ml CONCENTRADO_SALINO: 1530 ml EXTRACTO_ENERGETICO: 2000 ml AROMA_CITRICO: 1560 ml Coste total = 1107.0 centimos Azucar total = 700.4 g Sodio total = 4014.0 mg Cafeina total = 600.0 mg	1557

Apartado C)

i. A medida que vamos aumentando el volumen, tanto el número de soluciones como el tiempo de ejecución va aumentando, ya que sube el valor máximo de las distintas variables y por tanto el espacio de búsqueda es mayor. En nuestro caso, podemos ver que claramente el crecimiento no es polinómico, de hecho, cuando pasa de 5000 ml a 10000 ml (aumento del tamaño del problema por 2), el tiempo se multiplica por aproximadamente. Luego tomando que el tiempo $T = c \cdot V^n$, donde c es una constante y V es el volumen, tenemos que para $V_1 = 5000$, $V_2 = 10000$, $T_1 = 17739$ ms y $T_2 = 157000$ ms, $V_2/V_1 = 2$ y $T_2/T_1 = 8,85$ luego tenemos la ecuación $2^n = 8,85$ que nos da un $n = 3.15$, es decir, el algoritmo escala un poco peor que una función cúbica. Para 1 millón de ml, que es 100 veces más que 10000 ml, tenemos: $100^{3.15} = 2$ millones de veces más que con 10000 ml luego el tiempo será: $157000 \cdot 2$ millones = 314 millones de segundos que son unos 9,95 años, algo totalmente inasumible.

ii. De manera intuitiva, resolver un COP debería ser más lento, ya que no solo queremos resolver el problema, además queremos optimizarlo, en este caso, encontrar la solución con coste mínimo. Luego debemos buscar todas las soluciones y entre ellas la que tenga un menor coste. En la práctica, esta intuición se cumple si solo buscamos una solución del CSP, como es de esperar. Pero si buscamos todas las soluciones que resuelven el apartado a), este tarda mucho más que el apartado b), esto puede deberse a que al buscar soluciones con un coste mínimo, se puede mejorar la búsqueda descartando ramas que no lleven a mejorar la solución.

iii. Lo que sucede es que se obtiene un resultado en el que el total de monedas es un número float: 8,901. Esto se debe a que el número de monedas de cada tipo es un valor float, que es una cantidad real entre 0.0 y importe, y como con números reales hay infinitas combinaciones, encuentra una solución matemáticamente válida, pero que no es una solución del problema real. Aparte hay también errores de precisión, ya que tenemos fracciones de monedas, y el espacio de búsqueda pasa a ser mayor del que debería ser.

Ejercicio 2)

Explicación de la codificación empleada: En primer lugar, para organizar el modelo, hemos definido conjuntos del 1 al 6 y usado constantes con los nombres para que variables como hora[ALBA] se vean claras. La base del programa son tres arrays principales (proyecto, aula y hora) que deciden qué le toca a cada estudiante, usando all_different para garantizar que todo se asigne una única vez sin repetirse.

El truco clave de la codificación, es asociar cada proyecto, horario y aula al estudiante correspondiente, para ello se usa el predicado inverse para crear variables espejo (como est_de_proy), (para ver como hacerlo me ayudó la IA) de esta manera se evita usar bucles ineficientes cuando una restricción habla sobre un proyecto en lugar de un estudiante.

Finalmente, para el formato visual que se pedía, usamos arrays de texto junto con la función fix() para recuperar los resultados.

La única solución obtenida es:

Alba": "A* cuántico", "B1", "16:00"

Bruno": "Poda alfa-beta aplicada al encaje de bolillos", "B2", "15:00"

Carla": "Demostración matemática de que RTA* es un algoritmo absurdo", "A1", "18:00"

Diego": "Modelado de la incertidumbre a la hora de aprobar TSI sin estudiar nada", "A2", "17:00"

Elena": "Planificación automática jerárquica híbrida dedicada a la yuxtaposición de los mundos paralelos", "C1", "19:00"

Pablo": "Brazo robótico para jugar a la pocha", "C2", "20:00"

Ejercicio 3)

Explicación de la codificación empleada: Para modelar este problema tipo mochila (COP), hemos usado un enum para los objetos y un array de variables binarias (0..1) que definen su selección, resolviendo las dependencias técnicas y de

compatibilidad mediante inecuaciones lógicas simples. El aspecto más complejo fue aplicar el criterio lexicográfico, para el cual hemos usado una función ponderada a maximizar ($A * \text{utilidad} - B * \text{peso} - \text{coste}$). Para garantizar el orden estricto de prioridad, calculé las constantes A y B de forma que una sola unidad de mejora en el objetivo principal supere matemáticamente cualquier posible penalización combinada de los objetivos secundarios (Nota: el diseño de esta idea matemática se realizó consultando a una IA).

El problema tiene una sola solución óptima, que es la siguiente:

Objetos seleccionados:

- CAMARA (peso=2, coste=6, utilidad=9)
- ORDENADOR (peso=4, coste=10, utilidad=14)
- BATERIA_EXTRA (peso=2, coste=4, utilidad=7)
- BOTIQUIN (peso=3, coste=5, utilidad=8)
- RADIO (peso=4, coste=7, utilidad=11)

Utilidad total = 49

Peso total = 15

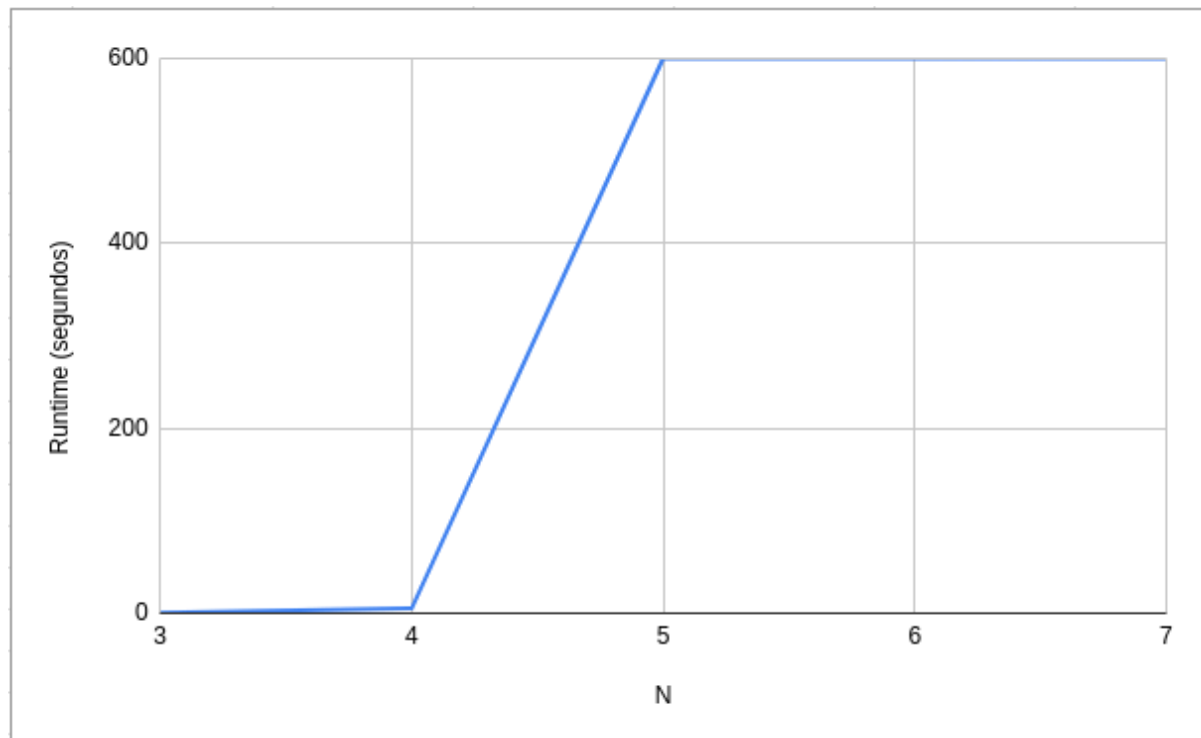
Coste total = 32

Ejercicio 4)

Explicación de la codificación empleada: Para codificar el problema hemos modelado el cuadrado mágico como una matriz $N \times N$ de variables enteras con dominio finito. Además, hemos reducido ese dominio imponiendo desde el principio: si N es par, solo se permiten valores pares, y si N es impar, solo impares, lo que reduce bastante el espacio de búsqueda. La restricción de cuadrado mágico se ha expresado forzando que todas las filas, columnas y las dos diagonales sumen el mismo valor, tomando como referencia la suma de la primera fila. Para exigir que al menos la mitad de las celdas contengan valores distintos, hemos usado la restricción global nvalue sobre la matriz aplanada. Por último, hemos quitado las simetrías geométricas (rotaciones y reflejos) comparando lexicográficamente la matriz original con sus 7 transformaciones equivalentes, de forma que el solver solo conserve una representante de cada clase de soluciones simétricas.

N	Núm. Valores Distintos	Suma Común	Suma Total Minimizada	Runtime
3	5	15	45	220 msec
4	8	24	96	5s 215 msec

5	-	-	-	infinito
6	-	-	-	infinito
7	-	-	-	infinito



Analizando estos resultados, podemos ver que el tiempo de ejecución no crece de forma proporcional al tamaño del tablero, ya que al aumentar de tamaño $N=3$ a $N=4$ pasa de ejecutarse en milisegundos a 5 segundos, y ya a partir de $N = 5$ tarda más de 5 minutos disparando hacia infinito, por lo que no se puede obtener una solución en tiempo razonable.

Por esta razón, podemos decir que no se trata de un problema escalable, ya que un sistema escalable es aquel que puede manejar incrementos en la carga de trabajo con un mantenimiento predecible del rendimiento. En este problema, al pasar de $N = 4$, a $N = 5$, el cuadrado pasa de tener 16 casillas a 25 casillas, pero el espacio de búsqueda se dispara.

Respecto a la clase de complejidad computacional, el problema del cuadrado mágico, planteado como un CSP, se trata de un problema NP, ya que se puede verificar en tiempo polinómico si una solución es correcta pero encontrar esa solución requiere explorar un árbol de búsqueda muy grande, por lo que no se puede hacer en tiempo polinómico. Además, como viene indicado en la presentación de la práctica, resolver CSP es NP-completo. Como la clase

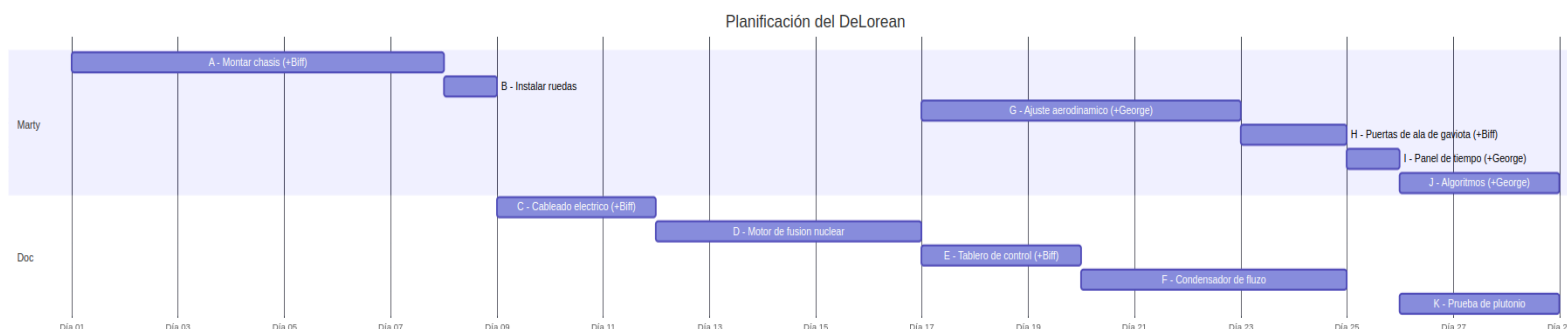
NP-completo (los problemas más difíciles dentro de NP) está reservada a los problemas de decisión (respuestas de 'sí o no'), y nuestro problema es un COP, ya que tenemos que minimizar la suma total, estamos ante un problema NP-duro, que son los problemas al menos tan difíciles como los NP-completos. Viendo los resultados comprobamos que efectivamente no se encuentra en la clase de complejidad P. Esto se evidencia claramente en la tabla, donde el tiempo de ejecución no crece de forma polinómica: mientras que para $N=3$ se resuelve en milisegundos, la explosión combinatoria propia de los problemas NP-Duros hace que para $N \geq 5$, el tiempo de búsqueda se vuelva inviable.

Ejercicio 5) Apartado A)

Explicación de la codificación empleada: Para modelar este problema hemos acotado el horizonte temporal a 84 días (el peor escenario posible) con el fin de reducir el espacio de búsqueda del solver. La asignación de tareas se ha optimizado usando un único array booleano (`es_marty`), y las duraciones base se han linealizado evitando sentencias if-else para mejorar la eficiencia computacional. La planificación temporal separa la duración original de la real para aplicar correctamente las ayudas de Biff y George, evitando el solapamiento de trabajadores y rompiendo simetrías al evaluar solo pares de tareas donde $t1 < t2$. Finalmente, el modelo se resuelve como un problema de optimización (COP) minimizando el instante de fin de la última tarea en lugar de la suma de duraciones, lo que fuerza al programa a paralelizar el trabajo al máximo.

DURACIÓN MÍNIMA de la construcción del DeLorean: 28 días.

Diagrama de Gantt con la planificación encontrada: En el diagrama podemos ver claramente cuando empieza y acaba cada tarea, y el solapamiento que hay entre las tareas que realizan Marty y Doc, que se da a partir del día 17. Antes no era posible debido a la necesidad de realizar las tareas predecesoras. Por ejemplo, para realizar la tarea G: Ajuste aerodinámico, previamente se tenían que completar la C y la D, y en ambas tarda menos tiempo Doc.



Apartado B)

En este apartado se pide obtener todas las configuraciones/planes diferentes que sean óptimas, para ello se coge el valor del tiempo mínimo obtenido de resolver el problema como un COP, guardar ese valor óptimo obtenido, y luego ejecutar un CSP imponiendo que el tiempo total coincida con el tiempo mínimo.

Una vez hecho esto, obtenemos que hay tres soluciones que cumplen que el tiempo de ensamblado del DeLorean sea de 28 días.

Ejercicio 6)

Explicación de la codificación empleada: Para optimizar el espacio de búsqueda del solver, hemos estructurado las variables de decisión de la siguiente manera: La matriz miembros une cada "asiento" del tribunal a un departamento específico mediante restricciones de dominio, evitando permutaciones entre profesores de distinto departamento. Además, asignamos el día y la sesión al tribunal en su conjunto y no a los profesores individuales, lo que simplifica drásticamente las restricciones de coincidencia temporal.

Para la asignación de proyectos, empleamos un mapeo inverso (tribunal_tfg) que garantiza que cada TFG sea evaluado por un único tribunal. Tras analizar el problema (12 TFGs y 3 tribunales), dedujimos que la carga estricta es de 4 TFGs por tribunal, implementándolo eficientemente con la restricción count. Las incompatibilidades de dirección, que un profesor que supervisado/dirigido un TFG no puede formar parte del tribunal que lo evalúe, se resuelven mediante evaluaciones $O(1)$ sobre la matriz booleana precalculada dirige. Finalmente, para reducir el espacio de búsqueda hemos eliminado las soluciones simétricas (resultados semánticamente idénticos pero con distinto orden de variables), introdujimos una restricción de orden lexicográfico (lex_lesseq), obligando al solver a ordenar los tribunales. También se han eliminado las simetrías por permutación de profesores dentro de un tribunal y por permutación de los TFGs, como se explica a continuación más detalladamente.

- ☐ ¿Cuál es el número de soluciones válidas obtenidas? En caso de ser 10 soluciones factibles o menos, lístelas explícitamente: El número de soluciones obtenidas ha sido 4, como se trata de menos de 10, las listamos:

Tribunal 1: "3"("DECSAI"), "5"("LSI"), "10"("ICAR") (Día: "Lunes", Sesión: "Mañana")

TFGs a evaluar: "1, 3, 5, 10"

Tribunal 2: "4"("DECSAI"), "6"("LSI"), "9"("ICAR") (Día: "Jueves", Sesión: "Tarde")

TFGs a evaluar: "7, 8, 9, 11"

Tribunal 3: "2"("DECSAI"), "7"("LSI"), "10"("ICAR") (Dia: "Viernes", Sesión: "Tarde")
TFGs a evaluar: "2, 4, 6, 12"

Tribunal 1: "3"("DECSAI"), "5"("LSI"), "10"("ICAR") (Dia: "Lunes", Sesión: "Mañana")
TFGs a evaluar: "1, 3, 5, 10"

Tribunal 2: "4"("DECSAI"), "6"("LSI"), "9"("ICAR") (Dia: "Jueves", Sesión: "Tarde")
TFGs a evaluar: "2, 8, 9, 11"

Tribunal 3: "2"("DECSAI"), "7"("LSI"), "10"("ICAR") (Dia: "Viernes", Sesión: "Tarde")
TFGs a evaluar: "4, 6, 7, 12"

Tribunal 1: "3"("DECSAI"), "5"("LSI"), "10"("ICAR") (Dia: "Lunes", Sesión: "Mañana")
TFGs a evaluar: "1, 3, 5, 10"

Tribunal 2: "4"("DECSAI"), "6"("LSI"), "9"("ICAR") (Dia: "Jueves", Sesión: "Tarde")
TFGs a evaluar: "2, 7, 9, 11"

Tribunal 3: "2"("DECSAI"), "7"("LSI"), "10"("ICAR") (Dia: "Viernes", Sesión: "Tarde")
TFGs a evaluar: "4, 6, 8, 12"

Tribunal 1: "3"("DECSAI"), "5"("LSI"), "10"("ICAR") (Dia: "Lunes", Sesión: "Mañana")
TFGs a evaluar: "3, 5, 6, 12"

Tribunal 2: "4"("DECSAI"), "6"("LSI"), "9"("ICAR") (Dia: "Jueves", Sesión: "Tarde")
TFGs a evaluar: "1, 9, 10, 11"

Tribunal 3: "2"("DECSAI"), "7"("LSI"), "10"("ICAR") (Dia: "Viernes", Sesión: "Tarde")
TFGs a evaluar: "2, 4, 7, 8"

Como podemos ver, en todas las soluciones, las diferencias reales radican en qué grupo de TFGs es evaluado por qué tribunal, cumpliendo siempre con las incompatibilidades por dirección.

☐ Discuta la posible existencia de soluciones simétricas:

Una solución simétrica en CSP ocurre cuando el solver devuelve múltiples soluciones que tienen valores matemáticamente distintos en sus variables, pero que representan exactamente la misma respuesta.

En nuestro caso la solución propuesta no tiene simetrías y para evitarlas hemos realizado lo siguiente:

1. Simetría por permutación de los Tribunales: Si el Tribunal 1 se reúne el lunes y el Tribunal 2 el jueves, una solución idéntica en la vida real es que el Tribunal 2 se reúna el lunes y el Tribunal 1 el jueves (simplemente intercambian "nombres"). Para solucionarlo se introdujo una restricción de orden lexicográfico (`lex_lesseq`) que obliga a que las características del Tribunal 1 sean siempre "menores o iguales" a las del Tribunal 2. Esto fuerza al solver a ordenar cronológicamente y por

ID de profesor los tribunales, eliminando todas las permutaciones equivalentes.

2. Simetría por permutación de profesores dentro de un tribunal: Un tribunal formado por los profesores [3, 5, 10] es exactamente igual a uno formado por [5, 3, 10]. Para solucionarlo en lugar de usar un array libre de 3 profesores, la matriz miembros[t, d] asocia directamente cada columna a un departamento (1=DECSAI, 2=LSI, 3=ICAR). Como los dominios están restringidos (DECSAI = 1..4, LSI = 5..7, etc.), el profesor 5 jamás puede intercambiar posición con el profesor 3, porque no se puede poner en su columna.
3. Simetría por permutación de los TFGs: Si un tribunal evalúa los TFGs [1, 3, 5, 10], una simetría sería que el tribunal evalúe los TFGs [3, 1, 5, 10]. Para solucionarlo la variable empleada es array[TFG] of var TRIB: tribunal_tfg. Es un mapeo inverso donde la variable es el TFG y el valor es el Tribunal. Un TFG sólo puede apuntar a un tribunal, eliminando la posibilidad de permutaciones.

Ejercicio 7)

Explicación de la codificación empleada: Para abordar este problema, hemos modelado la situación como un Problema de Satisfacción de Restricciones (CSP). Nuestra estrategia principal se basa en definir dos variables de decisión para cada partido: dia_partido[p] (con dominio de 1 a 7) y franja_partido[p] (con dominio 1=Tarde, 2=Noche, 3=Prime Time). Al asignar el día y la hora al partido, garantizamos que cada encuentro se programe exactamente una vez, evitando variables redundantes.

Nos hemos apoyado en varias estructuras auxiliares:

- Por un lado, la matriz booleana constante participa, que deduce automáticamente si un equipo juega en un partido concreto a partir de los datos de local y visitante.
- Por otro lado, introducimos las variables auxiliares juega[e,d] y b2b[e,d], vinculadas a las variables principales, lo que nos permite gestionar los descansos y la detección de back-to-backs (partidos en días consecutivos).

También evitamos la aparición de simetrías. Dado que los 12 partidos ya están claramente diferenciados por sus equipos y tanto los días como las franjas tienen un significado cronológico estricto, el solver no explorará ramas de búsqueda redundantes basadas en permutaciones.

- ☐ **Número de soluciones encontradas** (sin presentar simetrías): 12
- ☐ **Primera solución encontrada:**

CALENDARIO SEMANAL NBA (CSP)

=====

Back-to-backs totales = 2

Sesión	Día 1	Día 2	Día 3	Día 4	Día 5	Día 6	Día 7
Tarde	MIA-PHI / DEN-DAL	-	-	-	PHX-DEN	NYK-MIA / PHI-MIL	-
Noche	BOS-NYK	-	-	-	DAL-LAL / MIA-GSW	-	GSW-P HX
Prime Time	-	LAL-G SW	MIL-B OS	-	-	BOS-LAL	-

Leyenda:

BOS=Celtics, NYK=Knicks, PHI=Sixers, MIA=Heat, MIL=Bucks,
LAL=Lakers, GSW=Warriors, PHX=Suns, DEN=Nuggets, DAL=Mavericks